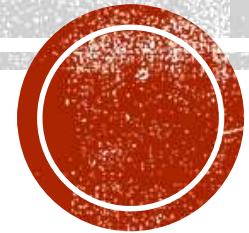


CSE327

SOFTWARE ENGINEERING

AKM iqtidar Newaz [IqN]

Lecture - 15



BUILDER DESIGN PATTERN

- ❑ Builder is a **creational design pattern** that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.
- ❑ For instance, let's open up a sandwich shop.
- ❑ We assume that Burgers, Hotdogs, and BLT are all types of sandwiches.
- ❑ Even though they're all sandwiches, but the composition of the sandwich is different.
- ❑ This can be represented with a [Sandwich class](#).



TRADITIONAL OOP

```
#pragma once
#include
<iostream>
#include <string>
#include <vector>
using namespace
std;
```

```
enum class Bread
{
    Toast,
    HotdogBun,
    BurgerBun
};
```

```
enum class Protein
{
    BeefPatty,
    Dog,
    Bacon
};
```

```
enum class Side
{
    Lettuce,
    Tomato,
    Onions,
    Cheese
};
```

sandwich.h

```
enum class Sauce
{
    Mayo,
    Ketchup,
    YumYum,
    Mustard
};
```

```
enum class Wrapper
{
    Paper,
    Box,
    Plastic
};
```

sandwich.h

```
class Sandwich
{
```

```
public:
```

```
    Sandwich() = default;
```

```
    Sandwich(Bread b, Protein p, vector<Side> sides, vector<Sauce> sauces, Wrapper w) :
    b{b}, p{p}, sides{sides}, sauces{saucers}, w{w}
    {
        cout << "A Sandwich is constructed" << endl;
    }
```

```
    Bread b;
    Protein p;
    vector<Side> sides;
    vector<Sauce> sauces;
    Wrapper w;
```

```
};
```

sandwich.h



TRADITIONAL OOP (OBJECT CREATION)

```
#include "sandwich.h"

int main()
{
    Sandwich burger(Bread::BurgerBun, Protein::BeefPatty,
        vector<Side>{Side::Lettuce, Side::Cheese, Side::Tomato},
        vector<Sauce>{Sauce::YumYum},
        Wrapper::Paper);

    Sandwich hotdog(Bread::HotdogBun, Protein::Dog,
        vector<Side>{Side::Onions},
        vector<Sauce>{Sauce::Ketchup, Sauce::Mustard},
        Wrapper::Box);

    Sandwich BLT(Bread::Toast, Protein::Bacon,
        vector<Side>{Side::Tomato, Side::Lettuce},
        vector<Sauce>{Sauce::Mayo}, Wrapper::Plastic);
}
```

noBuilderMain.cpp

Output:

A Sandwich is constructed
A Sandwich is constructed
A Sandwich is constructed



BUILDER PATTERN

```
#pragma once
#include "sandwich.h"
// Abstract builder class
class SandwichBuilder sandwichBuilder.h
{
public:
    virtual void AddBread() = 0;
    virtual void AddProtein() = 0;
    virtual void AddWrapper() = 0;
    virtual void LoadSides() = 0;
    virtual void DrizzleSauces() = 0;
    void AddSide(Side s)
    {
        product.sides.push_back(s);
    }
    void AddSauce(Sauce s)
    {
        product.sauces.push_back(s);
    }
    Sandwich ReturnProduct()
    {
        return product;
    }

    Sandwich product;
};
```

```
#pragma once
#include "sandwichBuilder.h"

class BurgerBuilder : public SandwichBuilder concreteBuilder.h
{
public:
    void AddBread() override
    {
        cout << "Heating up the Sesame Sprinked Bun" << endl;
        product.b = Bread::BurgerBun;
    }
    void AddProtein() override
    {
        cout << "Grilling the Wagyu" << endl;
        product.p = Protein::BeefPatty;
    }
    void AddWrapper() override
    {
        cout << "Wrapping with 100% recycled paper" << endl;
        product.w = Wrapper::Paper;
    }
    void LoadSides() override
    {
        cout << "Loading up the Burger" << endl;
        this->AddSide(Side::Lettuce);
        this->AddSide(Side::Tomato);
    }
    void DrizzleSauces() override
    {
        cout << "Drizzling the sauces" << endl;
        this->AddSauce(Sauce::YumYum);
    }
};
```



BUILDER PATTERN

```
class HotdogBuilder : public SandwichBuilder
{
public:
    void AddBread() override concreteBuilder.h
    {
        cout << "Steaming up the buns" << endl;
        product.b = Bread::HotdogBun;
    }
    void AddProtein() override
    {
        cout << "Boiling the hotdog in brine" <<
endl;
        product.p = Protein::Dog;
    }
    void AddWrapper() override
    {
        cout << "Placing the hotdog into box" <<
endl;
        product.w = Wrapper::Box;
    }
    void LoadSides() override
    {
        cout << "Loading up the Hotdog" << endl;
        this->AddSide(Side::Onions);
    }
    void DrizzleSauces() override
    {
        cout << "Drizzling the sauces" << endl;
        this->AddSauce(Sauce::Mustard);
    }
};
```

```
class BLTBuilder : public SandwichBuilder
{
public:
    void AddBread() override concreteBuilder.h
    {
        cout << "Toasting bread in toaster" << endl;
        product.b = Bread::Toast;
    }
    void AddProtein() override
    {
        cout << "Sizzling bacon in cast iron pan" << endl;
        product.p = Protein::Bacon;
    }
    void AddWrapper() override
    {
        cout << "Wrapping up in a compostable plastic wrap" << endl;
        product.w = Wrapper::Plastic;
    }
    void LoadSides() override
    {
        cout << "Loading up the BLT" << endl;
        this->AddSide(Side::Lettuce);
        this->AddSide(Side::Tomato);
    }
    void DrizzleSauces() override
    {
        cout << "Drizzling with the sauces" << endl;
        this->AddSauce(Sauce::Mayo);
    }
};
```



BUILDER PATTERN

```
#include "concreteBuilder.h"
```

```
Sandwich CreateSandwich(SandwichBuilder
*builder)
{
    builder->AddBread();
    builder->AddProtein();
    builder->LoadSides();
    builder->DrizzleSauces();
    builder->AddWrapper();
    return builder->ReturnProduct();
}

int main()
{
    cout << "Creating Bruger: -----" <<
endl;
    CreateSandwich(new BurgerBuilder());
    cout << "Creating Hotdog: ----- " <<
endl;
    CreateSandwich(new HotdogBuilder());
    cout << "Creating BLT: ---- " << endl;
    CreateSandwich(new BLTBuilder());
}
```

builderMain.cpp

Output:

Creating Bruger: -----
Heating up the Sesame Sprinked Bun
Grilling the Wagyu
Loading up the Burger
Drizzling the sauces
Wrapping with 100% recycled paper

Creating Hotdog: ----
Steaming up the buns
Boiling the hotdog in brine
Loading up the Hotdog
Drizzling the sauces
Placing the hotdog into box

Creating BLT: ----
Toasting bread in toaster
Sizzling bacon in cast iron pan
Loading up the BLT
Drizzling with the sauces
Wrapping up in a decompostable plastic wrap



METHOD CHAINING IN BUILDER

```
#pragma once
#include "sandwich.h"

// Abstract chain builder class
class ChainSandwichBuilder {
public:
    virtual ChainSandwichBuilder &AddBread() = 0;
    virtual ChainSandwichBuilder &AddProtein() = 0;
    virtual ChainSandwichBuilder &AddWrapper() = 0;
    virtual ChainSandwichBuilder &LoadSides() = 0;
    virtual ChainSandwichBuilder &DrizzleSauces() = 0;
    void AddSide(Side s)
    {
        product.sides.push_back(s);
    }
    void AddSauce(Sauce s)
    {
        product.sauces.push_back(s);
    }
    Sandwich ReturnProduct()
    {
        return product;
    }
    Sandwich product;
};
```

chainSandwichBuilder.h

We need to return an object reference so that the expression can continue to be chained onwards

```
#pragma once
#include "chainSandwichBuilder.h"

class BurgerBuilder : public ChainSandwichBuilder
{
public:
    ChainSandwichBuilder &AddBread()
    {
        cout << "Heating up the Sesame Sprinked Bun" << endl;
        product.b = Bread::BurgerBun;
        return *this;
    }
    ChainSandwichBuilder &AddProtein()
    {
        cout << "Grilling the Wagyu" << endl;
        product.p = Protein::BeefPatty;
        return *this;
    }
    ChainSandwichBuilder &AddWrapper()
    {
        cout << "Wrapping with 100% recycled paper" << endl;
        product.w = Wrapper::Paper;
        return *this;
    }
    ChainSandwichBuilder &LoadSides()
    {
        cout << "Loading up the Burger" << endl;
        this->AddSide(Side::Lettuce);
        this->AddSide(Side::Tomato);
        return *this;
    }
    ChainSandwichBuilder &DrizzleSauces()
    {
        cout << "Drizzling the sauces" << endl;
        this->AddSauce(Sauce::YumYum);
        return *this;
    }
};
```



METHOD CHAINING IN BUILDER

```
class HotdogBuilder : public ChainSandwichBuilder
{
public:
    ChainSandwichBuilder &AddBread()      concreteChainBuilder.h
    {
        cout << "Steaming up the buns" << endl;
        product.b = Bread::HotdogBun;
        return *this;
    }
    ChainSandwichBuilder &AddProtein()
    {
        cout << "Boiling the hotdog in brine" << endl;
        product.p = Protein::Dog;
        return *this;
    }
    ChainSandwichBuilder &AddWrapper()
    {
        cout << "Placing the hotdog into box" << endl;
        product.w = Wrapper::Box;
        return *this;
    }
    ChainSandwichBuilder &LoadSides()
    {
        cout << "Loading up the Hotdog" << endl;
        this->AddSide(Side::Onions);
        return *this;
    }
    ChainSandwichBuilder &DrizzleSauces()
    {
        cout << "Drizzling the sauces" << endl;
        this->AddSauce(Sauce::Mustard);
        return *this;
    }
};
```

```
class BLTBuilder : public ChainSandwichBuilder
{
public:
    ChainSandwichBuilder &AddBread()      concreteChainBuilder.h
    {
        cout << "Toasting bread in toaster" << endl;
        product.b = Bread::Toast;
        return *this;
    }
    ChainSandwichBuilder &AddProtein()
    {
        cout << "Sizzling bacon in cast iron pan" << endl;
        product.p = Protein::Bacon;
        return *this;
    }
    ChainSandwichBuilder &AddWrapper()
    {
        cout << "Wrapping up in a compostable plastic wrap" << endl;
        product.w = Wrapper::Plastic;
        return *this;
    }
    ChainSandwichBuilder &LoadSides()
    {
        cout << "Loading up the BLT" << endl;
        this->AddSide(Side::Lettuce);
        this->AddSide(Side::Tomato);
        return *this;
    }
    ChainSandwichBuilder &DrizzleSauces()
    {
        cout << "Drizzling with the sauces" << endl;
        this->AddSauce(Sauce::Mayo);
        return *this;
    }
};
```



METHOD CHAINING IN BUILDER

```
#include "concreteChainBuilder.h"
```

```
Sandwich CreateChainSandwich(ChainSandwichBuilder &builder)
{
    builder.AddBread().AddProtein().LoadSides().DrizzleSauces().AddWrapper();
    return builder.ReturnProduct();
}
```

```
int main()
{
    cout << "Creating Burger: -----" << endl;
    BurgerBuilder burgerBuilder;
    CreateChainSandwich(burgerBuilder); // Pass by reference

    cout << "Creating Hotdog: ----- " << endl;
    HotdogBuilder hotdogBuilder;
    CreateChainSandwich(hotdogBuilder);

    cout << "Creating BLT: ---- " << endl;
    BLTBuilder bltBuilder;
    CreateChainSandwich(bltBuilder);
}
```

builderChainMain.h

Output:

Creating Burger: -----
Heating up the Sesame Sprinked Bun
Grilling the Wagyu
Loading up the Burger
Drizzling the sauces
Wrapping with 100% recycled paper

Creating Hotdog: -----
Steaming up the buns
Boiling the hotdog in brine
Loading up the Hotdog
Drizzling the sauces
Placing the hotdog into box

Creating BLT: ----
Toasting bread in toaster
Sizzling bacon in cast iron pan
Loading up the BLT
Drizzling with the sauces
Wrapping up in a compostable plastic wrap



DIRECTOR CLASS

```
#pragma once
#include "concreteBuilder.h"

class Director
{
public:
    SandwichBuilder *builder;
    void SetBuilderType(SandwichBuilder *builder)
    {
        this->builder = builder;
    }
    // Dine In Order: AddBread -> AddProtein -> LoadSides -> DrizzleSauces
    Sandwich DineInOrder()
    {
        builder->AddBread();
        builder->AddProtein();
        builder->LoadSides();
        builder->DrizzleSauces();
        return builder->ReturnProduct();
    }
    // Takeout Order: Steps from Dine In Order -> AddWrapper
    Sandwich TakeoutOrder()
    {
        builder->AddBread();
        builder->AddProtein();
        builder->LoadSides();
        builder->DrizzleSauces();
        builder->AddWrapper();
        return builder->ReturnProduct();
    }
}
```

director.h

```
// Order without Sauces: AddBread -> AddProtein -> LoadSides
Sandwich NoSauce()
{
    builder->AddBread();
    builder->AddProtein();
    builder->DrizzleSauces();
    return builder->ReturnProduct();
}
// Order No Sides: AddBread -> AddProtein -> DrizzleSauces
Sandwich NoSides()
{
    builder->AddBread();
    builder->AddProtein();
    builder->DrizzleSauces();
    return builder->ReturnProduct();
}
};
```

- ❑ We can go further and extract a series of calls to the builder steps we use to construct a product into a separate class called director.
- ❑ The director class defines the order in which to execute the building steps, while the builder provides the implementation for those steps.



DIRECTOR CLASS

```
#include "director.h"
```

```
int main()
{
    Director d;
    Sandwich s;

    // Make Burger Orders
    d.SetBuilderType(new BurgerBuilder());
    s = d.TakeoutOrder();
    s = d.DineInOrder();
    s = d.NoSauce();
    s = d.NoSides();
    cout << "-----" << endl;
    // Make Hotdog Orders
    d.SetBuilderType(new HotdogBuilder());
    s = d.TakeoutOrder();
    s = d.DineInOrder();
    s = d.NoSauce();
    s = d.NoSides();
    cout << "-----" << endl;

    // Make BLT Orders
    d.SetBuilderType(new BLTBuilder());
    s = d.TakeoutOrder();
    s = d.DineInOrder();
    s = d.NoSauce();
    s = d.NoSides();
    return 0;
}
```



Output:

Heating up the Sesame Sprinked Bun
Grilling the Wagyu
Loading up the Burger
Drizzling the sauces
Wrapping with 100% recycled paper
Heating up the Sesame Sprinked Bun
Grilling the Wagyu
Loading up the Burger
Drizzling the sauces
Heating up the Sesame Sprinked Bun
Grilling the Wagyu
Drizzling the sauces
Heating up the Sesame Sprinked Bun
Grilling the Wagyu
Drizzling the sauces

Steaming up the buns
Boiling the hotdog in brine
Loading up the Hotdog
Drizzling the sauces
Placing the hotdog into box
Steaming up the buns
Boiling the hotdog in brine
Loading up the Hotdog
Drizzling the sauces
Steaming up the buns
Boiling the hotdog in brine
Drizzling the sauces
Steaming up the buns
Boiling the hotdog in brine
Drizzling the sauces

OUTPUT

Toasting bread in toaster
Sizzling bacon in cast iron pan
Loading up the BLT
Drizzling with the sauces
Wrapping up in a compostable plastic wrap
Toasting bread in toaster
Sizzling bacon in cast iron pan
Loading up the BLT
Drizzling with the sauces
Toasting bread in toaster
Sizzling bacon in cast iron pan
Drizzling with the sauces
Toasting bread in toaster
Sizzling bacon in cast iron pan
Drizzling with the sauces

